Manejo de Errores en Express

- Manejo de Errores en Express
  - Manejo de errores default
  - Manejo de errores en las rutas
  - Función manejadora de errores
  - Invocación de la función manejadora de errores
  - Ejemplo básico
  - Manejo de errores en promesas
  - Manejo en una estructura de carpetas `rutas+controladores`
  - Módulo `middlewares/errores.js`
  - Incluir módulo en `index.js`

Manejo de errores default

Los errores (síncronos) que generen nuestro servidor serán capturados por un manejador de errores incorporado
éste se encargará de dar una respuesta al cliente y el servidor seguirá ejecutándose:

```
const express = require ('express');
const app = express();
app.get('/usuario',(req,res)=>{
    if (tru) {                  //Reference error:tru is not defined, Express enviará error en html a cliente
        res.send("Respuesta Verdadera");
    } else {
        res.send("Respuesta Falsa");
    }
});
```

Mensaje de error en Html enviado al cliente:

```
Error: processing error in request at /products
    at /.../storefront/js/index.js:43:15
    at Layer.handle .. (/.../storefront/node_modules/express/lib/router/layer.js:95:5)
    at next (/.../storefront/node_modules/express/lib/router/route.js:137:13)
    at Route.dispatch (/.../storefront/node_modules/express/lib/router/route.js:112:3)
    at Layer.handle .. (/.../storefront/node_modules/express/lib/router/layer.js:95:5)
    at /.../storefront/node_modules/express/lib/router/index.js:281:22
    at Function.process_params (/.../storefront/node_modules/express/lib/router/index.js:341:12)
    at SendStream.error (/.../storefront/node_modules/serve-static/index.js:121:7)
    at SendStream.emit (node:events:390:28)
```

Manejo de errores en las rutas

Podemos manejar los errores en un nuestro servidor Express de dos maneras:
Implementando la lógica para detectarlos dentro de los manejadores de cada ruta pero esto no es recomendable ya que la función manejadora podría crecer mucho.

Validando si en la solicitud se incluye algun parametro o propiedad en el body:

```
app.post('/usuario', (req, resp) => {
  if(req.body.id == null){                                          // Verificar posibles errores

    response.status(400).json({ message: "No estas enviando el ID" }); // Enviar respuesta error
  }else{
    response.json({ mensaje: "Respuesta correcta"})                 // Envía respuesta ok
  }
})
```

Usando bloques de código try-catch:

```
app.get('/usuario', async (req, res) => {
  try {
         // Consulta a una Base de Datos
         // Consulta a una API

  } catch(error) { // Interceptar el error

    response.status(500)     // Regresar un mensaje de Error
            .json({ message: "Error de consulta a la BD/API" })
  }
})
```

Función manejadora de errores

Para manejar los errores que ocurran en un servidor Express usaremos una función middleware *especial*.
Esta función middleware recibirá cuatro parámetros, el parámetro adicional al inicio será para recibir el error
y deberá estar colocada después del manejo de rutas e implementar en su interior lógica para darle al front-end una respuesta adecuada.

```
//----------------------------
// Función Manejadora de Errores
//----------------------------
app.use((err,req,res,next)=>{
    console.log(err);
    res.status(500).json({
        status=err.status,
        mensaje=err.message,
    })
});
```

Invocación de la función manejadora de errores

Para poder invocar esta función middleware manejadora de errores, desde cualquier punto de nuestra aplicación usaremos el método **`next()`**, pero le tendremos que pasar el objeto del error, es decir: **`next(err)`**.

```
app.get('./usuario',(req,res,next)=>{

    let err = new Error('Ha ocurrido un error);
    next(err);
});
```

En nuestras funciones middleware o manejadores de ruta deberemos detectar los errores con bloques **`try-catch`** e invocar a la función **`next(err)`** con el error:

```
app.get('/usuario',(req,res,next)=>{
    try {
        //Consultar una API
        //Conexión-consulta a la Base de datos
        //Leet-escribir archivos en disco
    } catch(error) {
        next(error);
    }
});
```

Ejemplo básico

```
const express = require ('express');
const app = express();

app.get('/usuario',(req,res)=>{
    if (tru) {     // Reference error: tru is not defined  (Express enviará respuesta html al cliente)
        res.send("Respuesta Verdadera");
    } else {
        res.send("Respuesta Falsa");
    }
});

app.post('/usuario',(req,res,next)=>{      // Incluimos next en los parámetros de esta función middleware
    let err = new Error("Error Asíncrono");// Cuando en nuestro código asíncrono detectemos un error
    next(err);                             // debemos instanciar un Error e invocar la función next() con el error
});

// Al final de nuestras rutas deberemos incluir una función manejadora de errores
// Esta función recibe cuatro parámetros, el primero es el error que se envió con next(err)
app.use((err,req,res,next)=>{
    // dependiendo del error recibido deberemos implementar aquí la lógica para manejarlo
    // y regresar al cliente siempre una estructura uniforme o estándar informándole del error
    res.status(500).json({error:err.message});
});

app.listen(8080,(req,res)=>{ console.log("Servidor Express escuchando 8080"); });
```

Manejo de errores en promesas

Si en nuestro código asíncrono usamos promesas
por ejemplo una petición a una API usando la librería Axios.
Usaremos `next()` en el método `catch()` de la promesa

```
const express = require ('express');
const axios   = require ('axios');
const app     = express();

app.get('/usuario',  (request, response, next) => {

    axios.get("http://localhost:8080/catalogos")
        .then(response=>response.json)
            .then(jsonresponse=>response.send(jsonresponse))
                .catch(err => next(err))                    // Invocación de la función manejadora de error
})                                                          // en el método catch de la promesa

// Función Manejadora de Errores
app.use((err,req,res,next)=>{
    // Implementación de la lógica para manejar el error
    // y regresar error uniforme al cliente
    res.status(500).json({error:err.message});
});

app.listen(8080,(req,res)=>{ console.log("Servidor Express escuchando 8080"); });
```

Manejo en una estructura de carpetas `rutas+controladores`

Si nuestro servidor Express ya tiene una estructura de carpetas en donde definimos una carpeeta de **`/rutas`** una de **`/controladores`** y una de **`/vistas`** (opcional), se debe crear la función manejadora de errores en una módulo aparte de preferencia en una carpeta **`/middleware`** e importarla ese modulo en el archivo principal **`index.js`**

Carpeta	Contiene:
`./index.js`	Archivo principal de la app
`/routes/*.js`	Manejo de rutas
`/controllers/*.js`	Lógica de negocio
`/middleware/errores.js`	El archivo **`errores.js`** define y exporta la función manejadora de errores

Módulo `middlewares/errores.js`

```
function manejadorErrores(err, req, res, next) {

    res.status(500).json({ status: 0,
                           message: "Error interno del servidor",
     });
}

module.exports.manejadorErrores = manejadorErrores;
```

Incluir módulo en `index.js`

```
const express          = require("express");
const routes           = require("./routes");
const manejadorErrores = require("./middlewares/errores.js");

const app = express();

// Manejo de Rutas
app.use("/api", routes);

// Manejador de errores
app.use(manejadorErrores);

// Inicia Server
app.listen(3000, ()=>{
    console.log("Servidor escuchando en puerto 3000");
});
```